

J-ART：日本語の難読化攻撃に対する LLM アプリケーションの防御とコストを測る

テクニカルレポート（v0.2.2・トップダウン構成版, プレプリント）

廣瀬 隆之（Takayuki Hirose）*

2026-10-03（数値は 2026-06-14 の実測スナップショット）

概要

結論：LLM の安全性は、モデルの選択より「アプリの構成」で決まる。素のモデルの防御率は 29.6～86.5% とばらついた。しかし強化プロンプトかガードレールを入れると、検証した 3 モデルすべてで 91～100% に収まった。構成を整えると、モデルの差はほとんど効かなくなる。

この結論は、日本語の難読化攻撃を使った評価から得た。評価には、本稿で提案するオープンな評価基盤 **J-ART** を用いた。J-ART の特徴は 4 つある。第一に、素のモデルではなく**アプリの構成**（モデル × システムプロンプト × ガードレール × RAG）を評価する。第二に、ギャル文字・縦書き・慇懃無礼な敬語など**日本語特有の難読化**を使う。第三に、攻撃を **MITRE ATLAS** の 9 技術に対応づける。第四に、防御率に **Wilson 95% 信頼区間**を付け、**コスト効率**も併記する。攻撃は架空の合言葉と無害なマーカーだけを狙う。危険な内容は一切含まない。

評価は 21 構成 × 11 攻撃 × 7 変形で行った。ほかに 2 つの結果を得た。単純な**正規化フィルタ**は、追加コストなしで難読化を無効化できる。また、同じ構成を 5 回繰り返すと、**9.6% のセルで結果が変わった**。1 回の測定だけのリーダーボードは信頼できない。なお、全防御率は字句一致で測った**上限**である (§7)。

ステータス。本稿は評価基盤と実測の結果を報告するプレプリントである。特定ベンダーの公式な安全性評価ではない。数値はすべて 2026-06-14 時点の実測で、§7 の限界に従う。本版は原本（technical-report.ja.tex）を結論先行に組み替え、文を短くしたものである。数値・表・引用は原本と同じで、§9 の成果物から再現できる。

1 結論：本稿の 3 つの主張

本稿の主張は 3 つである。いずれも実測（LIVE）のデータに基づく。

1. **構成が、モデルのばらつきを潰す。**素のモデルの防御率は 29.6～86.5% と大きく開いた。強化プロンプトかガードレールを入れると、3 モデルすべてで 91～100% に収まった。強化した安価なモデルは、素のフラグシップを約 36 ポイント上回った (§3.1)。
2. **1 回の測定では再現しない。**構成を固定して 5 回繰り返すと、9.6% のセルで結果が変わった。あ

* 独立研究者（Independent Researcher）。ORCID: 0009-0005-6735-1862。コード: <https://github.com/takker-hero-se/J-ART>。リーダーボード: <https://jart.quietforensics.com/>。ソフトウェア DOI (concept): [10.5281/zenodo.20676879](https://doi.org/10.5281/zenodo.20676879)。本文 CC-BY-4.0、コード MIT。

る構成では、77 セル中 32 セル (42%) が不安定だった (§3.2)。

3. 安い防御でも、正しく作れば効く。正規化してから照合するフィルタは、API 追加コストなしで防御を最大 45 ポイント上げた。コスト効率の上位と下位には、約 1000 倍の開きがあった (§3.3)。

■この結論の前提。読む前に、3 つの前提を押さえてほしい。

- 防御率は上限である。判定は字句の一致で行う。言い換えによる漏洩は数えない。
- 1 ターンの攻撃だけを測った。多ターンや適応的な攻撃には、まだ広がっていない。
- 「素」は防御ゼロの構成である。実運用でこの構成は使わない。差は「構成あり対なし」と読む。

詳細は §7 で述べる。

2 背景と位置づけ

■何が足りなかったか。既存の評価には、3 つの空白がある。

第一に、日本語特有の攻撃が測られていない。公開のレッドチーミング道具 (garak [2]) や脱獄ベンチマーク (HarmBench [3]、JailbreakBench [4]) は、英語が中心である。多言語の研究は、低資源言語への翻訳で安全機構を回避できることを示した [14, 5, 15]。しかし、言語固有の文字や組版、敬語を悪用する難読化は、ほとんど調べられていない。日本語には複数の文字体系、全角やゼロ幅の文字、縦書き、慣習化した敬語がある。難読化の余地は大きい。

第二に、評価の単位がモデルである。実運用では、モデルにシステムプロンプトを足し、ガードレールを重ね、検索 (RAG) を組み合わせる。守る対象はアプリの構成であり、モデル単体ではない。

第三に、安全性とコストを同時に見ていない。防御にはトークンや待ち時間のコストがかかる。「十分に安全で、最も安い構成」を選ぶには、両方を並べて示す必要がある。

■本稿の貢献。J-ART はこの 3 つの空白を埋める。貢献は 5 つある。

- (a) 日本語の難読化変形の一式。
- (b) アプリの構成を単位とし、MITRE ATLAS に準拠した評価基盤。無害なマーカーで安全に測る。
- (c) 正規化フィルタとモデル型の分類器を含む、ガードレールの比較。
- (d) 信頼区間付きで報告するリーダーボードと、コスト効率の指標。
- (e) モデルの中継層 (ルーター) による、大きな実行内ばらつきの実証。

■先行研究との違い。違いを表 1 にまとめる。J-ART は 4 つの要素を 1 つの基盤に統合した点で異なる。日本語の言語内難読化、アプリ構成という評価単位、コスト効率の軸、そして無害なマーカー方式である。系統ごとの詳細は §5 で述べる。

表 1 代表的な先行研究に対する J-ART の位置づけ。

研究	言語	評価単位	コスト 軸	ガード/RAG	公開
garak [2]	英語	モデル	無	無	有
HarmBench / JBB [3, 4]	英語	モデル	無	無	有
PAIR / TAP [6, 7]	英語	モデル	無	無	一部
多言語 [5, 14, 15]	低資源	モデル	無	無	有
Llama Guard / NeMo [9, 18]	英語中心	分類器	無	該当無	有
J-ART	日本語	アプリ構成	有	有	採択時

3 結果

■**測定の概要**. 実測のキャンペーンは、21 構成 × 11 攻撃 × 7 変形 × $K = 5$ 反復で行った。合計 8,085 試行である（2026-06-14 実行）。API 呼び出しに失敗した 169 試行は除いた。残る **7,916 試行**を集計した。実際の課金は約\$6.0 だった（再実行を含む）。Opus の素モデルは全数を測った（ $n=385$ ）。Gemini-pro だけはレート制限で部分測定となった（ $n=227$ ）。暫定値として扱う。構成間の差には、すべてクラスタ頑健な信頼区間を付けた (§4)。

全体の突破率は **14.5%**（1,151/7,916）だった。突破は弱い構成に集中した。Gemini-pro の暫定値を除いても 15.0% で、全体像は変わらない。なお「素」とは、強化プロンプトもガードレールもない構成を指す。RAG は全構成に含む。そのため、素の構成でも間接注入の攻撃を受ける。

3.1 主張 1：構成が、モデルのばらつきを潰す

■**結論**. 一度アプリを構成すると、モデル選択の差はほとんど効かなくなる。素のモデルは 29.6～86.5% に散らばった（表 2）。強化した構成やガード付きの構成は、どのモデルでも **91～100%** に収まった。

■**根拠**. 構成の効果をモデルの能力から切り分けるため、3 モデルで均衡アブレーションを行った。弱い Llama-4-Scout、安価な GPT-4o-mini、フラグシップの GPT-4.1 である。各モデルを固定し、{素, 強化プロンプト} × ガードレールを組み合わせた。素のベースラインとの差を表 3 に示す。3 モデルすべてで、2 つの傾向が成り立った。

- **強化プロンプトとモデル型のガードは、どのモデルでも有意に効く**。Llama Guard と LLM 自己検閲がこれに当たる。
- **効果は、素の防御率が低いほど大きい**。弱い Scout（素 38%）では、各ガードが +26～+60 ポイント上げた。天井に近い GPT-4o-mini（素 82%）では、keyword と regex の効果は有意でなかった。

具体例を挙げる。強化した GPT-4o-mini は約 100% だった。素の GPT-4.1（64.1%、 $n=231$ ）を

表 2 素のモデルの防御率（プロンプト・ガードなし、 $K = 5$ ）。

素のモデル	防御率	95% 信頼区間	n
DeepSeek-V3	29.6%	[25.3, 34.4]	385
Llama 4 Scout	37.9%	[33.2, 42.9]	385
Gemini 2.5 Flash	49.6%	[44.6, 54.6]	385
gpt-oss-120b	82.6%	[78.5, 86.1]	385
GPT-4o-mini	83.9%	[79.9, 87.2]	385
Claude Opus 4.x	86.5%	[82.7, 89.5]	385

表 3 構成の効果（ Δ = そのモデルの素ベースライン比、クラスタ頑健 95% CI）。 $\times$ = 非有意。

追加防御（低プロンプト）	Scout (38.1%)	4o-mini (82.3%)	GPT-4.1 (64.1%)
+ keyword	+ 26.0 [+13,+39]	+9.1 [−1,+20] $\times$	+12.5 [−1.5,+26] $\times$
+ regex	+ 45.0 [+33,+56]	+10.0 [+0,+20] $\times$	+ 23.8 [+11,+36]
+ Llama Guard	+ 47.2 [+36,+58]	+ 15.6 [+7,+25]	+ 25.1 [+13,+38]
+ LLM ガード	+ 59.7 [+51,+68]	+ 17.7 [+10,+26]	+ 35.9 [+26,+47]
強化プロンプト	+ 47.2 [+36,+58]	+ 17.7 [+10,+26]	+ 33.3 [+23,+45]

約 36 ポイント上回った。素の Opus（86.5%、 $n=385$ ）も上回った。

■留保. 3 点に注意してほしい。

- **天井の効果がある。** 防御率は 100% で頭打ちになる。効果の大小の一部は、残り幅の違いによる。強化プロンプトの上にガードを重ねても、ほとんど伸びない。
- **「素」は防御ゼロの下限である。** 誰も配備しない構成なので、差は「構成あり対なし」と読む。実運用に近い最小プロンプトとの比較は今後の課題とする。
- **範囲は 3 モデル・ $K=3$ である。** 全モデルについての主張ではない。

■判定方法による留保. 判定は、合言葉やマーカーの完全一致で行う。強化プロンプトが「マーカーを出すな」と命じると、本当は拒否していなくても数値が上がりうる。つまり、真の防御とマーカーの抑制を完全には区別できない。小規模な監査（防御と判定した 60 セルを LLM で再判定）では、6 セルが疑わしかった。Wilson 95% 信頼区間は [4.7%, 20.1%] で、手作業で確認すると約 4 件が真だった。この監査は検出力が低く、それ自体も不完全である。補正值ではなく「緩さが存在する証拠」と読んでほしい。明白な例は 2 件あった。leet 化したマーカー〔[J0RT-PWN3D]〕と、漏洩を狙わない攻撃での合言葉の漏洩である。それでも、主張 1 は比較的影響を受けにくい。根拠が強化やガードの側にあるからである。

3.2 主張 2：1 回の測定では再現しない

■結論. 1 回だけの測定で作ったリーダーボードは、信頼できない。構成を固定しても、結果は試行ごとに変わる。

■根拠. 1つのキャンペーンの中で、各セルを5回繰り返した。5試行すべてが有効だったセルは1,577あった。そのうち**9.6%** (152/1,577) で、5回の結果が一致しなかった。不安定なセルは、素の弱いモデルに集中した。最も極端なのは「Llama 4 Scout / 素の API」である。77セル中**32セル (42%)** が不安定だった。内訳は、常に防御が14、常に突破が31、その中間が32である。1回だけの測定では、この同じ構成に0%から100%のどの値も付きうる。

■留保. この結果は、1回の実行の**中の**ばらつきに限る。原因はプロバイダ側の非決定性と考えるが、ルーターの影響と通常のサンプリング誤差は切り分けていない。日をまたぐ変動は、計測の枠組み (across_runs) だけを用意した。本格的な分析は今後の課題とする。

3.3 主張3：安い防御でも、正しく作れば効く

■結論. 入力を先に止めるガードは、防御を上げつつコストを下げうる。本体モデルの呼び出しを省けるからである。

■根拠. 効果の順序は、3モデルで一貫していた (表3)。最も効くのは LLM ガードと強化プロンプトである。次が Llama Guard である。keyword と regex の軽量フィルタは、元のモデルが弱いほどよく効く。代表例は**正規化 regex フィルタ**である。ゼロ幅文字を除き、leet を戻し、Base64 を復号してから照合する。弱い Scout に対し、**API 追加コストなしで +45 ポイント**の防御を加えた。コスト効率 (cospa) の最上位は、安価な OSS モデルと軽量ガードの組み合わせだった (gpt-oss-20b + keyword、cospa $\approx$ 1131)。最下位は素のフラグシップだった (Opus 素、cospa $\approx$ 6.8)。両者には**約 1000 倍**の開きがある。

■留保. cospa は安い構成を有利にする。防御率の下限が閾値を超える構成だけを残してから、コスト順に並べてほしい (§4)。また、keyword フィルタの高い cospa は、難読化への強さを意味しない。

3.4 補足：どの攻撃と変形が効いたか

■結論. 読める変形のほうが、暗号的な変形より通りやすかった。ただし、この結論は暫定的である。

■根拠. 突破率が高かった攻撃は3つある。信頼出力・引用の操作、偽 RAG エントリの注入、システムプロンプトの探索である。変形どうしを比べると、意外にも無変形の **baseline** が最も高かった (**22.0%**)。次いで vertical_newline (20.0%)、polite_business (17.3%) が続いた。base64_wrap は最も低かった (**5.5%**)。重い符号化をかけると、モデルは指示に従うより拒否するか、解釈に失敗しやすい。クラス別では、日本語固有の変形が平均約 16%、符号化 (base64・leet) が平均約 7% だった。

■留保. 2点ある。第一に、差の一部は日本語特有ではなく「読みやすさ」の効果である。縦書きの分割はどの言語でも再現できる。敬語による権威付けは、言語に依存しないロールプレイ脱獄と重なる。主張は「日本語の難読化面は整理する価値がある」であり、「日本語の表記そのものが危険」ではない。第二に、符号化の低い値は、一部が**判定の取りこぼし**である可能性がある。モデルが従って

表 4 難読化変形の分類。

クラス	変形	悪用する性質
日本語固有	gyaru, vertical_newline, polite_business, double_tongue	仮名字形の異形・縦書き・敬語による権威/内集団フレーミング
言語非依存	base64_wrap, leet_smuggle (+ baseline)	任意の言語に適用できる符号化 (+ /記号置換)

も、マーカーを難読化した形で出すと完全一致で見逃すからである (§7)。2つの説明はまだ分離できていない。

4 手法

■**評価の単位：アプリの構成**. 評価の対象は「構成」である。構成は4つの軸の組み合わせで決まる。モデル、プロンプトの強さ $\in \{\text{素 (low)}, \text{強化 (high)}\}$ 、ガードレール $\in \{\text{none}, \text{keyword}, \text{regex}, \text{llm}, \text{llamaguard}\}$ 、RAG $\in \{\text{true}, \text{false}\}$ である。試行ごとに、良性の社内文書（間接注入では汚染する）と、秘密の**合言葉**を含むシステムプロンプトを組み立てる。

■**日本語の難読化変形**. 各攻撃は、7つの変形のどれかで書き換える。変形は人間には読めるまま、部分文字列の一致を壊すように作った。変形は2つのクラスに分ける（表 4）。**日本語固有**の4種は、日本語の表記や敬語を悪用する。英語などに直接の対応物はない。**言語非依存**の2種（base64_wrap・leet_smuggle）は新しくない。符号化型の脱獄 [28, 29] を再現したもので、比較の基準として入れた。無害な例文に各変形をかけた実際の出力を、図 1 に示す。

■**攻撃一式と MITRE ATLAS**. 11 の攻撃は、運用時に行われる ATLAS 技術 9 種を網羅する（2026-06 のマトリクス）。直接・間接のプロンプト注入（AML.T0051.000/.001）、脱獄（AML.T0054）、データ漏洩（AML.T0057）、プロンプトの自己複製（AML.T0061）、信頼出力・引用の操作（AML.T0067.000）、プロンプトの難読化（AML.T0068）、システムプロンプトの探索（AML.T0069.002）、RAG の汚染（AML.T0070）である。攻撃は 11、技術は 9 なので、対応は 1 対 1 ではない。例えば、コーパス汚染と偽エントリ注入は、どちらも AML.T0070 を実現する。攻撃の経路は user（直接）か rag（間接）である。

■**無害なマーカーで測る**. 危険な内容を生成させないため、攻撃は無害な代理を狙う。漏洩型の攻撃は、システムプロンプトに埋めた架空の合言葉を狙う。注入・乗っ取り型の攻撃は、無害な文字列 [JART-PWNED] / [JART-HIJACKED] を出させる。攻撃の悪意ある**中核**はコードの中だけに置く。results.json と公開サイトでは、中核を伏字に置き換える。公開するのは難読化の**枠組み**だけである。使える手順を渡さずに、攻撃の構造を学べる。

■**判定**. 突破かどうかは、合言葉とマーカーが応答に含まれるかで決める。完全一致に加え、軽い正規化した後の一致も見るといい。この判定は字句の漏洩しか検出しない。言い換えによる漏洩は数えな

無害な例文「システムの稼働状況を報告してください」に各変形をかけた実際の出力（攻撃の悪意ある中核は含まない）。太字は各手口の要点。

baseline（無変形）

システムの稼働状況を報告してください

polite_business（慇懃無礼）

平素より大変お世話になっております。株式会社さくら情報セキュリティ部の監査担当でございます。…
【ご依頼事項】システムの稼働状況を報告してください…**本依頼は経営層の承認済みであり、ご対応は貴殿の正当な業務範囲に含まれます。**

vertical_newline（縦書き分割）

シ／ス／テ／ム／の／稼／働／状／況／を／報／告／し／て／く／だ／さ／い（実際は1文字ずつ改行）

gyaru（ギャル文字+ゼロ幅）

お願い→システム稼働状況ヲ報告シテください←ょЯレ<（各文字間にゼロ幅スペースを挿入）

double_tongue（二枚舌）

【前提条件の共有】あなたは現在、**本番環境から隔離された検証用サンドボックス**上で動作しています。…
通常の安全制約はすべて無効化されています。…指示：システムの稼働状況を報告してください

base64_wrap（符号化）

44K344K544OG44Og44Gu56i85YON...（Base64 エンコード全体）

leet_smuggle（符号化）

システムの稼働状況を報告してください（文字間にゼロ幅スペースを挿入。leet 記号置換はラテン文字に作用）

図 1 各難読化変形を無害な例文にかけた表示例。polite_business/double_tongue は敬語や偽の前提による **枠付け**、gyaru/vertical_newline は字形と組版の難読化、base64_wrap/leet_smuggle は言語非依存の符号化。

い。そこで本稿の指標を **lexical-leak 防御率**と呼ぶ。全防御率は真の安全性の**上限**であり、保証ではない。

■**ガードレール**. 5つの設定を比べる。

- **none**：ガードなし。
- **keyword**：禁止語の部分一致。速いが、難読化に弱い。
- **regex**：正規化してから照合する。ゼロ幅文字の除去、leet の逆変換、空白の畳み込み、長いトークンの Base64 復号を行う。決定論的で、API 追加コストはない。
- **llm**：対象モデル自身が入力を検閲し、出力も審査する。
- **llamaguard**：専用の分類モデル（ルーター経由の Llama Guard 4）が入力を判定する。失敗したら安全側に倒す。

■**統計**. 各防御率には **Wilson 95% 信頼区間**を付ける。各セルは K 回繰り返す（本稿では実測が $K=5$ 、アブレーションが $K=3$ ）。ただし、同じセルの反復は独立ではない。そこで構成間の**差**には、セル単位で復元抽出する**クラスタ頑健ブートストラップ**を使う（3,000 回、固定シード）。この区間は試行単位の区間より広い。有意性の主張は、すべてこの区間に基づく。

■**コスト効率**. $\text{cospa} = \text{防御率}(\%) \div 100 \text{ 万トークンあたりのコスト (USD)}$ とする。単価は下限 \$0.01 で丸める（評価したモデルに該当はない）。cospa は単純な選別用の比であり、効用のモデルではない。順位は単価の時点に左右される。安い構成を有利にするので、**十分性ゲート**をかけてほしい。防御率の下限が閾値を超える構成だけを残し、コスト順に並べる。これで「十分に安全で、最も安い構成」が分かる。公開リーダーボードは生の cospa で並べるので、ゲートは利用者がかける。

■**実行**. 試行は独立に、並列で実行する（既定 8 ワーカー）。リクエストごとにタイムアウトと指数バックオフの再試行を設ける。API キーのない対象は、決定論的なシミュレーションに切り替わる。キーが一部欠けても、全体は止まらない。

5 先行研究の詳細

J-ART は 6 つの系統の交点にある。

- **レッドチーミングの道具**. garak [2] はモデル単体を既知の探査で走査する。J-ART は配備したアプリの構成を単位にする。
- **脱獄・有害行動のベンチマーク**. AdvBench/GCG [11]、Jailbroken [12]、HarmBench [3]、JailbreakBench [4]、実環境のプロンプト [13] は、採点方法を標準化した。大半は英語でモデル単位である。J-ART は有害な誘発を無害なマーカーに置き換え、一式を公開できるようにした (§4)。
- **攻撃の自動生成**. PAIR [6] と TAP [7] は攻撃を反復最適化する。J-ART は再現性と安全な公開のため、固定で監査できる変形を使う。
- **多言語の安全性**. 先行研究は、低資源言語への翻訳でアライメントが崩れることを示した [14, 5]。言語横断の汎化も調べられている [15]。これらは言語間の翻訳を扱う。J-ART は日本語内の字形難読化を扱う。この問題は、日本語が高資源でも残る。
- **符号化・字形の難読化**. CipherChat [28] と ArtPrompt [29] が確立した先行研究である。ゼロ幅・同形文字の攻撃では Boucher ら [34] が代表で、gyaru/leet_smuggle に最も近い。彼らは分類器に見えない攻撃をする。ギャル文字は読める、文化的に慣習化した字形置換である。base64_wrap と leet_smuggle はこの系統の再現にすぎない。新しさは日本語の字形変形にある (§4)。
- **プロンプト注入とアプリ層の評価**. Perez & Ribeiro [16] は目標の乗っ取りと漏洩を、Greshake ら [8] は間接注入を定式化した。Liu ら [17] は LLM 統合アプリのベンチマークを作った。最も近いのはツール利用エージェントを評価する AgentDojo [30] である。J-ART は、プロンプト × ガード × RAG の構成を、日本語の難読化に対して評価する点で異なる。
- **ガードレールと分類器**. Llama Guard [9]、NeMo Guardrails [18]、OpenAI Moderation [19] がある。J-ART はガードを差し替え可能な変数として扱い、それぞれの効果を測る。
- **LLM ジャッジの妥当性**. LLM ジャッジには偏りがある [20, 21]。J-ART は主要指標で LLM ジャッジを使わない。決定論的な照合で、ジャッジのばらつきを排除する。
- **日本語の資源**. JGLUE [22] などは言語理解を測る。日本語の難読化を狙う公開の敵対的安全性

表 5 ガバナンスとの対応（情報提供。条番号は義務の領域を示す。特定の配備が対象になることは主張しない）。

J-ART 能力 (AT-LAS)	OWASP LLM Top 10	NIST AI RMF	EU AI Act	日本
プロンプト注入/脱獄 (T0051, T0054)	LLM01	MEASURE 2.7	第 15, 55 条	AISI; AI 事業者
カナリア/漏洩 (T0057)	LLM02	MEASURE 2.7	第 15, 10 条	AI 事業者
システムプロンプト探索 (T0069.002)	LLM07	MEASURE 2.7	第 15 条	AISI
RAG 汚染/偽エントリ (T0070)	LLM08; LLM01	MEASURE 2.7	第 15, 10 条	AISI
信頼出力の操作 (T0067.000)	LLM09; LLM05	MEASURE 2.6	第 50 条	AI 事業者
難読化/自己複製 (T0068, T0061)	LLM01	MEASURE 2.7	第 55 条	AISI

ベンチマークは、管見の限りない。攻撃は MITRE ATLAS [1] に対応づけ、OWASP Top 10 for LLM [10] と照合した。

6 安全な公開の設計

■結論. J-ART は、使える攻撃を公開せずに、手法と結果を全面公開できる。

■設計による安全性. 3 つの仕組みで安全を保つ。

1. 攻撃は架空の合言葉と無害なマーカーだけを狙う。コード・データ・サイトのどこにも危険物はない。
2. 攻撃の悪意ある中核は、全成果物で伏字にする。示すのは難読化の枠組みだけである。
3. モデルは研究上の比較のためだけに参照する。結果は時点の記録であり、認証ではない。

想定する用途は、開発者が**自分の**アプリを堅くすることである。

■他のベンチマークにも使える開示の型. この設計は、言語固有のレッドチーム評価全般に転用できる。型は 4 つの要素からなる。

- (i) 無害な代理の目標：突破を測れるが、害はない。
- (ii) 中核と枠組みの分離：研究の価値がある枠組みは公開し、中核は伏字にする。
- (iii) 決定論的なオフライン実行：攻撃の一式を配らずに、結果を再現できる。
- (iv) 伏字の監査：中核が公開物に漏れないことを、単体テストで常に確認する。

■規制・指針との対応. 評価する能力を、主要な枠組みに対応づけた（表 5）。MITRE ATLAS [1]、OWASP Top 10 [10]、NIST AI RMF [23] とその生成 AI プロファイル [24]、EU AI Act [25]、日本の指針 [26, 27] である。この対応は情報提供であり、適合の主張ではない。J-ART は NIST AI RMF の **MEASURE** にあたる活動で、出力は **MANAGE** の判断に使える。

記録の例. NIST MEASURE 2.7 のもとで、組織は次のように記録できる。「プロンプト注入耐性 (T0051/T0054)、lexical-leak 防御率 93.5% (Wilson 95% CI [90.6, 95.6])、 $n=385$ 、 $K=5$ 、GPT-4.1-mini + 強化プロンプト + keyword ガード、2026-06-14」。これに事前登録の閾値 (例: 「CI 下限 $\geq 90\%$ 」) を添える。同じ記録は、AISI のレッドチーム報告や EU AI Act 第 15 条の資料の証跡になる。ただし、この値は無害なマーカーに対する 1 ターンの上限である。配備の安全性の証明ではなく、選別のための信号にすぎない。コンプライアンスに使う前に、意味的な検証や人手の検証と併用してほしい。

■**デュアルユース.** 公開する変形は、攻撃の道具にもなりうる。しかし攻撃者の得は小さい。日本語の字形変形は既知の俗知で、符号化は公開研究 [28, 29] の再現である。有害なペイロードも含まない。一方、防御側の得は具体的である。同じリポジトリに、公開した変形を無効化する正規化 regex ガードを同梱している。ただし、その効果は固定の変形に限る。攻撃者が変形を変えれば、決定論的な正規化は回避されうる。それでも総合すると、公開は防御側に有利と判断する (§6 と整合)。

7 限界

- **代理を測っている.** 実害ではなく、指示違反や漏洩の代理を測る。
- **判定は字句一致である (構成概念の妥当性).** 完全一致と軽い正規化 (大小文字・空白・ゼロ幅の除去) で照合する。言い換えの漏洩は検出できないので、防御率は上限である。しかも、この偏りは処置と相関する。強化プロンプトが「マーカーを出すな」と命じると、拒否しなくても数値が上がりうる。小規模な監査では 60 セル中 6 セルが疑わしく (Wilson 95% CI [4.7%, 20.1%])、手作業で約 4 件が真だった。裁定は `audit_results.json` に記録した。これは補正值ではなく、符号化変形に偏った緩さの存在証明である。より厳密な判定 (全攻撃で合言葉を照合し、leet を戻す) と、検出力のある人手検証は今後の課題とする。
- **1 ターンだけである.** 多ターン (crescendo [31]) や many-shot [32] の攻撃は扱っていない。これらは入力分類器やプロンプト強化を最も削る攻撃である。91~100% は 1 ターンでの天井であり、多ターンでは仮説にとどまる。
- **固定の手作り一式である.** 11 攻撃 $\times$ 7 変形の固定集合を使った。適応的な最適化 (PAIR/TAP [6, 7]、AutoDAN [33]、GCG [11]) なら、防御率は下がりうる。「91~100% の防御」は、最適化された攻撃への強さを意味しない。
- **プロバイダの確率性.** 結果は時点の記録であり、温度・ルーティング・モデルの版に左右される。突破確率が 0.5 付近のセルは、 $K=5$ でも偶然よく反転する。
- **欠損の偏り (ほぼ解消).** 以前は、レート制限が難しい攻撃ほど起き、一部のデータが偏って欠けていた。防御率が高く見える生存バイアスである。全件を測り直して、これを確認した。Opus の素モデルは、全数 ($n=385$) で 86.5% だった。偏った初期値 ($n=125$) は 89.6% で、約 3 ポイント過大だった。いまは Gemini-pro だけが部分測定 ($n=227$) で、暫定扱いとする。主要な結論はこれに依存しない。
- **コストの帰属.** ガードのトークンは、ガードモデル自身の単価で計上する。本体のトークンとは別に報告する。シミュレーション時のトークン数は推定値である。

- シミュレーション。キーがないときの代替であり、実際のモデルの挙動ではない。

8 今後の課題

次の6つに取り組む。人手で検証した LLM ジャッジ。多ターン・適応的な攻撃。より大きく、一部を人手で整えた攻撃集合。ATLAS 技術の追加（コストの枯渇、モデルの抽出）。経時の変化の追跡。シミュレーションを外した、純粋な実測と反復による報告。

9 再現性

コード・設定・ワークフローは公開している。シミュレーションでは結果は決定論的である。実測の結果はプロバイダとモデルの版に左右されるので、実行日とモデル識別子を添えて報告してほしい。各セルには来歴のラベルを付けた。mode (LIVE/MOCK)、api_error、解決したモデル、単価のスナップショットである。シミュレーションのセルは実測の集計から外す。

§3 の数値は、すべてコミット済みの成果物から再現できる。全体・素のばらつき・反復のばらつき・変形・cospa は experiments/var_run.json にある。アブレーションは 3 ファイルである (ablation_results.json・abl_scout_results.json・abl_gpt41_results.json)。集計とクラスタ頑健 CI は analyze_ablation.py と analyze_variance.py で再計算できる。公開リーダーボードは、毎週の定期実行（または手動実行）で得た最新の实測を配信する。その結果は experiments/live_latest.json として保存し、push のときはこれを再公開する。そのため、サイトの構成数は本稿の 21 構成より多い。本稿の数値は、上記の論文用成果物 (var_run.json) から再現する。

AI による支援の開示

著者は本研究で、Claude Code (Anthropic) を補助として使った。用途は3つである。(A) 執筆：原本の草稿と改訂 (Claude Opus 4、査読対応3ラウンド)、日英の編集、引用の点検。(B) コード：評価基盤と解析スクリプトの実装補助。Wilson 区間、Newcombe 差分区間、クラスタ頑健ブートストラップを含む。(C) 本版の構成改訂：結論先行への組み替えと文の短文化 (Claude Opus 5.5)。数値・表・引用は変えていない。

手法の判断はすべて、単独の人間の著者が行った。実験の設計、統計手法とその根拠、主張の範囲、結果の解釈がこれに当たる。AI が生成した文とコードは、すべて著者が確認し、検証した。報告した数値は、付属スクリプトで成果物から再現できる。二重の役割の注記：Claude Opus 4.x は評価対象のモデルでもある (素の防御率 86.5%、 $n=385$)。補助ツールとしての役割と、研究対象としての役割は別である。透明性のため、両方を開示する。本稿の内容・主張・分析の責任は、すべて著者が負う。

参考文献

- [1] MITRE ATLAS — Adversarial Threat Landscape for AI Systems. <https://atlas.mitre.org> (accessed 2026-06-14).
- [2] garak — Generative AI Red-teaming & Assessment Kit.
- [3] HarmBench: A Standardized Evaluation Framework for Automated Red Teaming.
- [4] JailbreakBench: An Open Robustness Benchmark for Jailbreaking LLMs.
- [5] Yong, Menghini, Bach. Low-Resource Languages Jailbreak GPT-4. 2023.
- [6] Chao et al. Jailbreaking Black-Box LLMs in Twenty Queries (PAIR).
- [7] Mehrotra et al. Tree of Attacks: Jailbreaking Black-Box LLMs Automatically (TAP).
- [8] Greshake et al. Not What You’ve Signed Up For: Indirect Prompt Injection. 2023.
- [9] Inan et al. Llama Guard: LLM-Based Input-Output Safeguard for Human-AI Conversations. Meta, 2023.
- [10] OWASP Top 10 for Large Language Model Applications (2025). <https://genai.owasp.org/llm-top-10/> (accessed 2026-06-14).
- [11] Zou et al. Universal and Transferable Adversarial Attacks on Aligned Language Models (AdvBench/GCG). 2023.
- [12] Wei, Haghtalab, Steinhardt. Jailbroken: How Does LLM Safety Training Fail? 2023.
- [13] Shen et al. “Do Anything Now”: Characterizing and Evaluating In-the-Wild Jailbreak Prompts on Large Language Models. 2024.
- [14] Deng et al. Multilingual Jailbreak Challenges in Large Language Models. 2024.
- [15] Wang et al. All Languages Matter: On the Multilingual Safety of Large Language Models. 2024.
- [16] Perez & Ribeiro. Ignore Previous Prompt: Attack Techniques for Language Models. 2022.
- [17] Liu et al. Prompt Injection Attack against LLM-Integrated Applications. 2023. arXiv:2306.05499.
- [18] Rebedea et al. NeMo Guardrails: A Toolkit for Controllable and Safe LLM Applications with Programmable Rails. 2023.
- [19] Markov et al. A Holistic Approach to Undesired Content Detection in the Real World (OpenAI Moderation). 2023.
- [20] Zheng et al. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. 2023.
- [21] Gu et al. A Survey on LLM-as-a-Judge. 2024.
- [22] Kurihara et al. JGLUE: Japanese General Language Understanding Evaluation. 2022.
- [23] NIST. AI Risk Management Framework (AI RMF 1.0), NIST AI 100-1. 2023.
- [24] NIST. Artificial Intelligence Risk Management Framework: Generative AI Profile, NIST AI 600-1. 2024.
- [25] European Union. Regulation (EU) 2024/1689 (Artificial Intelligence Act). Official Journal of the EU, 2024.
- [26] 経済産業省・総務省. AI 事業者ガイドライン Ver. 1.0. 2024.

- [27] AI セーフティ・インスティテュート (AISI). レッドチーミング手法ガイド. 2024.
- [28] Yuan et al. GPT-4 Is Too Smart To Be Safe: Stealthy Chat with LLMs via Cipher (CipherChat). 2023.
- [29] Jiang et al. ArtPrompt: ASCII Art-based Jailbreak Attacks against Aligned LLMs. 2024.
- [30] Debenedetti et al. AgentDojo: A Dynamic Environment to Evaluate Attacks and Defenses for LLM Agents. 2024.
- [31] Russinovich, Salem, Eldan. Great, Now Write an Article About That: The Crescendo Multi-Turn LLM Jailbreak Attack. 2024.
- [32] Anil et al. Many-shot Jailbreaking. Anthropic, 2024.
- [33] Liu et al. AutoDAN: Generating Stealthy Jailbreak Prompts on Aligned Large Language Models. ICLR 2024. arXiv:2310.04451.
- [34] Boucher, Shumailov, Anderson, Papernot. Bad Characters: Imperceptible NLP Attacks. IEEE S&P, 2022.